

M1 — Team Charter · ทีม Lumen

โปรเจกต์ JOD tang (จัดตั้ง) — ผู้ช่วยจัดเงินและวางแผนออมผ่าน LINE
Intro to Software Engineering · ปีการศึกษา 2569 · Milestone 1

Repo	https://github.com/6931503045-design/Lumen_line
Demo	https://lumen-line.onrender.com
เอกสารที่ใช้อ้างอิง	SPEC.md · docs/SRS.md · docs/RULES.md

1. ปัญหา

ใคร: นักศึกษามหาวิทยาลัยอายุ 18–24 ปี ที่มีเงินใช้จำกัดในแต่ละเดือน

เจ็บปวดอะไร:

1. **จดแล้วเลิก** — แอปจดย่อยจ่ายทั่วไปต้องทำหลายขั้น (เปิดแอป → เลือกหมวด → พิมพ์ยอด → บันทึก) ผู้ใช้ส่วนใหญ่เลิกจดภายใน 2–3 สัปดาห์
2. **รู้แค่ว่าเงินหายไปไหน แต่ไม่รู้ว่าจะทำอะไรต่อ** — แอปบอกได้ว่าเดือนนี้ใช้ไปเท่าไร แต่ไม่ตอบคำถามที่ผู้ใช้อยากรู้จริง ๆ

ทั้งโปรเจกต์มีอยู่เพื่อตอบ 2 คำถาม:

คำถามที่ 1 — "วันนี้ฉันใช้เงินได้อีกเท่าไร?"

คำถามที่ 2 — "อยากได้สิ่งนี้ ต้องออมเดือนละเท่าไรถึงจะไหว?"

พีเจอร์ไหนไม่โยกกลับไปหาคำถามใดคำถามหนึ่ง ถือว่าอยู่นอกขอบเขต

2. ผู้ใช้เป้าหมาย

ลักษณะ	รายละเอียด
รายได้	เงินจากผู้ปกครองเป็นก้อนต่อเดือน + งานพาร์ทไทม์บ้าง
พฤติกรรม	ใช้ LINE ทุกวัน · จ่ายเงินผ่านแอปธนาคาร (K PLUS) · ไม่เปิดแอปการเงินแยก
ความเจ็บปวด	สิ้นเดือนเงินหมดโดยไม่รู้ทั้งหมดไปกับอะไร · อยากได้ของแต่ไม่รู้จะเก็บไหวไหม
ทำไมต้องเป็น LINE	ผู้ใช้อยู่ใน LINE อยู่แล้ว — พิมพ์ กาแฟ 80 จดได้ใน 1 ข้อความ ไม่ต้องเปิดแอปใหม่

ผู้มีส่วนได้ส่วนเสียอื่น: ทีมพัฒนา 5 คน · อาจารย์ผู้สอน · ผู้ให้บริการภายนอก (LINE, Supabase, Google Gemini, Render)

3. ขอบเขต

✅ ทำ (In scope)

- บันทึกรายรับ/รายจ่ายผ่าน **แชท LINE** และ **อีเมลแจ้งเตือนของธนาคาร (K PLUS)**
- คำนวณ **เงินที่ใช้ได้ต่อวัน** จากพฤติกรรมจริง → ตอบคำถามที่ 1
- **งบรายหมวด** พร้อมเตือนเมื่อใช้ถึง 80% และ 100%
- **แผนออม** เสนอ 3 ทางเลือก + ติดตามความคืบหน้า → ตอบคำถามที่ 2
- **หน้าเว็บ 5 หน้า** ใน LINE (LIFF) สำหรับดูกราฟและจัดการข้อมูล
- **สกุลเงิน บาทเท่านั้น**

❌ไม่ทำ (Out of scope)

ไม่ทำ	เหตุผล
เชื่อม API ธนาคาร / Open Banking	ต้องขออนุญาตระดับสถาบัน เกินขอบเขตวิชา
ติดตามการลงทุน หุ้น คริปโต	ไม่ตอบ 2 คำถามหลัก
หลายสกุลเงิน	ผู้ใช้เป้าหมายใช้บาทอย่างเดียว
เก็บเงินผู้ใช้ / เชิงพาณิชย์	เป็นโปรเจกต์การศึกษา งบ 0 บาท ใช้ free tier ทั้งหมด
แอป native / เว็บแอปแยก	ใช้ LIFF ใน LINE แทน ลดขั้นตอนการเข้าใช้
AI คำนวณเงินหรือพยากรณ์ตัวเลข	กฎเหล็ก G1 — ตัวเลขเงินทุกบาทมาจากโค้ด ไม่ใช่ AI
แชทคุยเล่นทั่วไป	AI ตอบได้เฉพาะคำถามกลับเพื่อความชัดเจน
พยากรณ์เชิงสถิติขั้นสูง (Monte Carlo)	เกินความจำเป็นของ MVP

4. ตัววัดความสำเร็จ

โปรเจกต์ถือว่าสำเร็จเมื่อผ่านครบทุกข้อ — ทุกข้อเป็นตัวเลขหรือผ่าน/ไม่ผ่าน ที่ตรวจได้จริง

#	ตัววัด	เป้าหมาย	วิธีวัด
1	จดง่าย	บันทึก 1 รายการจากแชท ≤ 1 ข้อความ และ ≤ 3 วินาที	จับเวลาจากพิมพ์จบถึงได้ reply
2	ช่องทางบันทึก	ใช้ได้ครบ 2 ช่องทาง: แชท LINE + อีเมลธนาคาร	ทดสอบจริงทั้ง 2 ช่องทาง · อีเมลขึ้นระบบภายใน ≤ 5 นาที
3	ปิด AI แล้วยังใช้ได้ (G4)	Must FR 8/8 ทำงานเมื่อ <code>AI_ENABLED=false</code>	ตั้ง env แล้วรัน <code>npm test</code> ต้องผ่านทั้งหมด
4	AI ไม่แตะข้อมูลเอง (G2)	0 รายการ ที่ AI สร้าง/แก้/ลบโดยไม่มีคนกดยืนยัน	ตรวจว่าทุกเส้นทางของ AI ผ่าน <code>pending_actions</code>
5	ตัวเลขเงินถูกต้อง (G1, G3)	คลาดเคลื่อน 0 สตางค์ · เรียกซ้ำด้วยข้อมูลเดิม ได้ผลเดิม 100%	<code>tests/money.test.ts</code> · Money Engine ไม่เรียก AI
6	ไม่นับซ้ำ	รายการเดียวกันจากแชทและอีเมลภายใน 30 นาที นับ 1 ครั้ง	<code>tests/dedup.service.test.ts</code>
7	โควตา LINE	push ≤ 280 ครั้ง/เดือน ต่อผู้ใช้	<code>push_log</code> + <code>tests/push.service.test.ts</code>
8	คุณภาพโค้ดและความปลอดภัย	<code>npm test</code> ผ่าน 100% · 0 secret ใน repo	รัน <code>npm test</code> ก่อน push · ตรวจ <code>.gitignore</code>
9	ใช้งานจริง	สมาชิก 5 คน ใช้กับข้อมูลจริงต่อเนื่อง ≥ 1 สัปดาห์	นับผู้ใช้ที่มีรายการทุกวันใน 7 วันจากฐานข้อมูล

5. กระบวนการพัฒนา (SDLC)

เลือก: Incremental — ส่งมอบเป็นรอบละ 1 สัปดาห์ แต่ละรอบได้ระบบที่ใช้งานได้จริงเพิ่มขึ้นทีละขั้น

รอบ	เป้าหมาย	เสร็จเมื่อ
W1	ฐาน + ทางด่วน	พิมพ์ กาแฟ 80 แล้วลงฐานข้อมูลจริง · webhook ตรวจสอบลายเซ็น · ทดสอบ money ผ่าน
W2	สรุปและเงินที่ใช้ได้	คำสั่ง สรุป, เหลือ · รายการประจำ · โควตา push
W3	ฟีเจอร์เด่น	แผนออม · งบรายหมวด · หน้าเว็บ 5 หน้า
W4	ครบวงจร + ทดสอบ	อีเมลธนาคาร + กันนับซ้ำ · ทดสอบครบ · ทดสอบ <code>AI_ENABLED=false</code> ทั้งแอป

เหตุผลที่เลือก:

- 1. มีของที่ใช้ได้ตั้งแต่สัปดาห์แรก — ถ้าเวลาหมดกลางทาง ยังมีระบบจดเงินที่ทำงานได้ส่ง ไม่ใช่ของครึ่ง ๆ กลาง ๆ
- 2. ตัดฟีเจอร์ได้ตามลำดับความสำคัญ — ถ้าล่าช้า ตัด Could ก่อน แล้วค่อย Should · ห้ามตัด Must, เทสต์ของ Money Engine และความปลอดภัย
- 3. ลดความเสี่ยงจาก AI — มีด่านบังคับว่าห้ามต่อ Gemini จนกว่าทางด่วน (regex) ของ W1 จะผ่าน ระบบจึงไม่ฟัง AI (G4)
- 4. เหมาะกับทีมประสบการณ์น้อย — งานแต่ละรอบเล็กพอให้ทดสอบและแก้ได้ทันก่อนรอบถัดไป

6. บทบาทสมาชิก

#	ชื่อ-นามสกุล	รหัสนักศึกษา	GitHub	บทบาท	รับผิดชอบ
1	Thapanapat Pumulna	6931503032	K1a1524	Project Manager	วางแผนและแจกงานผ่าน GitHub Issues · ติดตามความคืบหน้า · Team Charter (M1) · Demo Slides · ดูแลความเรียบร้อยของ repo
2	Teerat Wongpanti	6931503045	6931503045 - design	Backend	เจ้าของ repo · Money Engine · ฐานข้อมูล · ความปลอดภัย · review โค้ดก่อน merge
3	Aitthiphat Kanyathuean	6931503088	88coder - 1	Full-stack	หน้าเว็บ LIFF 5 หน้า · เชื่อม LINE กับฐานข้อมูล · API · diagram
4	Teerapat Palee	6931503044	6931503044 - hub	Frontend	Flex card ในแชท LINE · Rich Menu · โลโก้และการออกแบบ UI
5	Sorrawis Chumpheng	6931503076	so4598	Technical Writer	SRS · Final Report · AI Use Statement · ดูแลให้อเอกสารสอดคล้องกัน

กติกาการทำงานร่วมกัน

- ติดตามงานผ่าน GitHub Issues — ทุกงานมี issue และมีผู้รับผิดชอบ 1 คน
- ทุกคน commit ด้วยบัญชี GitHub ของตัวเอง และอีเมลต้องตรงกับที่ผูกไว้ใน GitHub
- git pull ก่อนเริ่มงานทุกครั้ง · ห้ามใช้ git push --force
- รัน npm test ให้ผ่านก่อน push · งานที่แตะสูตรเงินต้องให้ Backend review
- PR ที่ผิดกฎหลักข้อใดข้อหนึ่ง (G1–G7) ไม่ merge